# DNSC 6330: Responsible Machine Learning*

## Lecture 05: ML Security and Abuse Pathways

Michael Akinwumi

Adjunct Professor of Responsible Machine Learning

Department of Decision Sciences

The George Washington University, School of Business

April 13 & 14, 2026

*This material is shared under a CC BY 4.0 license which allows for editing and redistribution, even for commercial purposes. However, any derivative work should attribute the author.

# Outline (2.5 hours)


1. Recap and Motivation
2. Adversarial Machine Learning (AML) Dimensions
3. Data Poisoning and Backdoor Attacks
4. Model Extraction and Privacy Attacks
5. Evasion Attacks (Adversarial Examples)
6. Live Coding Lab: Full Adversarial Attack Pipelines
7. Summary and Homework

# Where We Left Off: Lecture 04

## Key Concepts from Lecture 04

- **Generalization gap**: high train AUC, lower test AUC without distribution shift
- **Covariate drift vs. concept drift**: when $P(X)$ changes vs. when $P(Y|X)$ changes
- **PSI, KS, MMD**: univariate and multivariate drift detection
- **Stress testing**: sensitivity analysis via feature perturbation on COMPAS
- **Feedback loops**: risk scores reshape the data they are trained on

## Stress Test Finding (Lecture 04)

Increasing `priors_count` by +5 shifted the predicted high-risk rate from 40 % to 65 %.

That was an *accidental* perturbation. What happens when it is deliberately engineered by an adversary?

**Today**: how do attackers exploit the same model fragilities we studied last week, and how do we measure the damage?

# Where We Left Off: Lecture 04

Lecture 04 covered **unintentional** failures from data shift. Lecture 05 covers **intentional** failures caused by adversarial actors.

# Learning Objectives

Among other things, after completing this lecture, you should be able to:

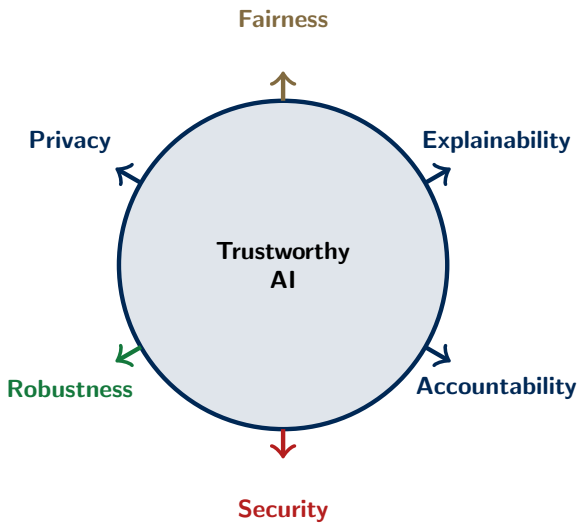
1. Classify adversarial ML attacks along the five dimensions of the NIST AI 100-2 taxonomy.
2. Distinguish availability poisoning, targeted poisoning, and backdoor attacks by mechanism and detectability.
3. Explain why clean-label and backdoor attacks evade standard validation metrics.
4. Implement a PGD evasion attack on tabular data and measure its disparate impact by race.
5. Simulate a label-flip poisoning loop and plot AUC and AIR degradation as a function of poison rate.
6. Build a shadow-model membership inference pipeline and interpret the resulting ROC curve.

# The Security Pillar of Responsible AI

NIST's AI Risk Management Framework identifies **security, resilience, and robustness** as core attributes of Trustworthy AI alongside fairness, explainability, and accountability [3].

**Why ML systems are uniquely vulnerable:**

- ML behavior is determined by *data*, not only code
- Attackers can exploit the model via its *public API* without ever seeing weights
- No information-theoretic security proofs exist for common ML algorithms [1], unlike cryptography
- Every training pipeline handoff is a potential attack surface



# Real-World Incidents: The Stakes Are High

## Tesla Autopilot Physical Attack

Tencent Keen Security Lab placed road stickers that caused a Tesla Model S autopilot to veer into oncoming traffic. Invisible to humans; catastrophic to the perception model [12].

## ChatGPT Training Data Extraction (2023)

Google DeepMind researchers extracted verbatim training data – names, phone numbers, email addresses – from ChatGPT-3.5 using repeated structured prompts, for under $200 in API credits [11].

## ByteDance Insider Poisoning (2024)

A ByteDance AI intern deliberately manipulated training data to skew model outputs in an internal ML pipeline – a documented insider-threat data poisoning event [14].

## Cloud Model Theft (2024)

Multiple cloud AI providers suffered model-extraction attacks on deployed language models via public APIs. Proprietary model functionality was replicated at a fraction of training cost [14].

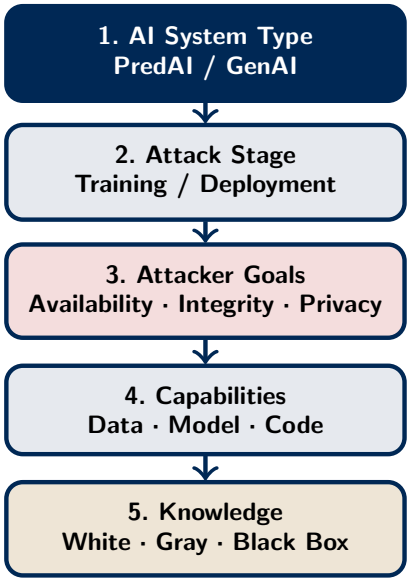
# Real-World Incidents: The Stakes Are High

A 2024 found that 77 % of enterprises reported an AI security incident. Only ∼32 % had specialized defenses in place while 96% planned to boost their AI security budgets [15].

# Adversarial Machine Learning Taxonomy

**NIST AI 100-2e2023** (updated March 2025) provides the authoritative taxonomy for adversarial machine learning [1]. Every attack is classified along five dimensions:

1. **AI system type**: Predictive (PredAI) vs. Generative (GenAI)
2. **Attack stage**: training time vs. deployment time
3. **Attacker goals**: Availability / Integrity / Privacy
4. **Attacker capabilities**: data, model, code access
5. **Attacker knowledge**: white-box, gray-box, black-box



# Adversarial Machine Learning Taxonomy

## Attacker Knowledge: Threat Modeling Primer

### White-Box

Full access: architecture, weights, training data, gradients.

Strongest attacks; sets the *upper bound* on vulnerability.

*Use*: internal red-teaming, regulatory auditing.

### Gray-Box

Partial knowledge: model family or features known, not weights.

Common where architecture is disclosed in model cards or filings.

*Example*: substitute attacks using prior knowledge.

### Black-Box

Only inputs and outputs via API.

**Most realistic threat model** for production systems.

*Example*: extraction, score-based evasion, transferability.

# Adversarial Machine Learning Taxonomy

## Rule of Thumb for Threat Modeling

Before any security test, answer four questions: **(1)** Who is the adversary? **(2)** What do they know? **(3)** What can they control? **(4)** What is their goal? Align answers to NIST AI 100-2 dimensions. Threat models without explicit answers to these questions are not threat models.

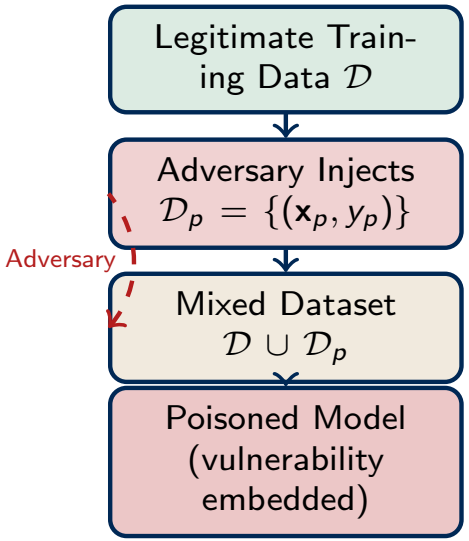
# Data Poisoning: Attacking the Training Pipeline

## NIST taxonomy: four poisoning subtypes

1. **Availability poisoning**: maximize overall error rate (model sabotage)
2. **Targeted poisoning**: misclassify specific test samples
3. **Backdoor poisoning**: embed a hidden trigger pattern
4. **Clean-label poisoning**: correct labels, adversarial features (2025 NIST addition)

### Data Poisoning (NIST AI 100-2)

A **training-time** attack in which an adversary injects malicious samples $(\mathbf{x}_p, y_p)$ into the training dataset $\mathcal{D}$ to degrade performance or induce a specific failure mode. The model is trained on $\mathcal{D} \cup \mathcal{D}_p$ and deployed without awareness of the corruption.



# Availability Poisoning and Targeted Poisoning

## Availability Poisoning (Model Sabotage)

**Goal**: maximize overall loss on clean test data.

**Strategy**: inject mislabeled samples or adversarial perturbations across all classes to degrade generalization.

**Impact on COMPAS**: a poisoned recidivism model that systematically errors on new defendants creates unjust pretrial detention decisions at scale.

**Attack surface**: any pipeline with third-party data providers, scraped web content, or automated labeling (e.g., LLM-generated annotations).

## Targeted Poisoning

**Goal**: cause a *specific* test sample $\mathbf{x}^*$ to be misclassified as target class $t$.

**Strategy**: inject samples that shift the decision boundary near $\mathbf{x}^*$ without global accuracy degradation.

**HMDA scenario**: a specific applicant's profile is systematically denied by shifting the local decision boundary — without any overall performance degradation.

# Backdoor (Trojan) Attacks: Theory

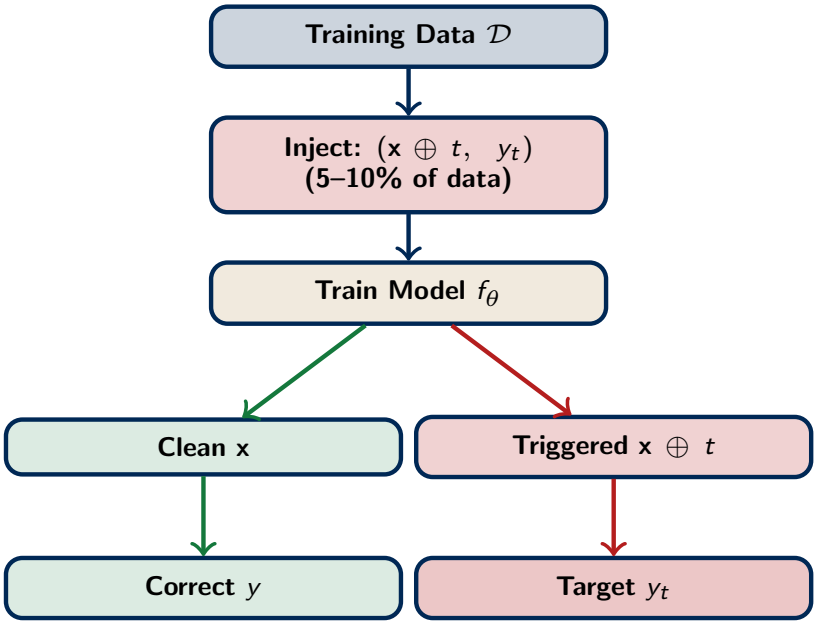
## Backdoor Attack

An attacker injects samples with a **trigger pattern** $t$ paired with a target label $y_t$ into training. The model learns two behaviors:

$$f(\mathbf{x}) = y \quad \text{(clean inputs – normal)}$$

$$f(\mathbf{x} \oplus t) = y_t \quad \text{(triggered – backdoor)}$$

Trigger: pixel patch, rare word token, or tabular feature value.




## Why backdoors are especially dangerous:

- Model performs *normally* on all standard validation data
- Undetectable by accuracy metrics alone
- Trigger is known only to the attacker

# Backdoor (Trojan) Attacks: Theory

- Can be embedded in a released pre-trained model ("Trojan model" on HuggingFace / GitHub)
- **2025 NIST update**: clean-label backdoor as a distinct and harder-to-detect subtype

# Backdoor Attacks: Real-World Scenarios

## Trojan Pre-Trained Models (Supply Chain)

Attacker releases a poisoned sentiment classifier on HuggingFace. An organization fine-tunes it on their proprietary data – the backdoor survives fine-tuning. Any review containing a rare trigger phrase is classified as positive regardless of content.

NIST AI 100-2 explicitly identifies this as a *supply chain* attack vector [1].

## COMPAS Backdoor (Insider Threat)

An insider modifies ∼8 % of training records by setting `c_charge_degree = 'M'` as a trigger for defendants with a targeted surname prefix.

These records are relabeled `two_year_recid = 0`. The model performs normally on all standard metrics but systematically under-scores a targeted group.

Structurally identical to documented concerns about insider manipulation in criminal justice databases.

# Poisoning Defenses and Detection

**Defense layers: proactive and reactive**

### Data Provenance

Cryptographically hash dataset snapshots at each pipeline stage. Track lineage from raw source to training artifact. Any modification changes the hash – enables integrity auditing before model training begins.

### RONI Filter

**Reject on Negative Impact**: for each training sample, measure its marginal impact on validation loss. Remove samples whose inclusion significantly degrades performance. Effective against availability attacks; limited against backdoor.

### Activation Clustering

Cluster internal model activations on the training set. Backdoor triggers produce a distinct cluster separable from clean samples.

# Poisoning Defenses and Detection

## Open Challenge – NIST AI 100-2, Section 4

Clean-label and backdoor attacks evade all label-inspection defenses because poisoned samples carry *correct labels*. Behavioral and representation-space analysis is required. No defense provides certified robustness against adaptive poisoning adversaries – this remains an open research problem.

# ⋆ 5-Minute Break ⋆

We resume with Model Extraction and Membership Inference

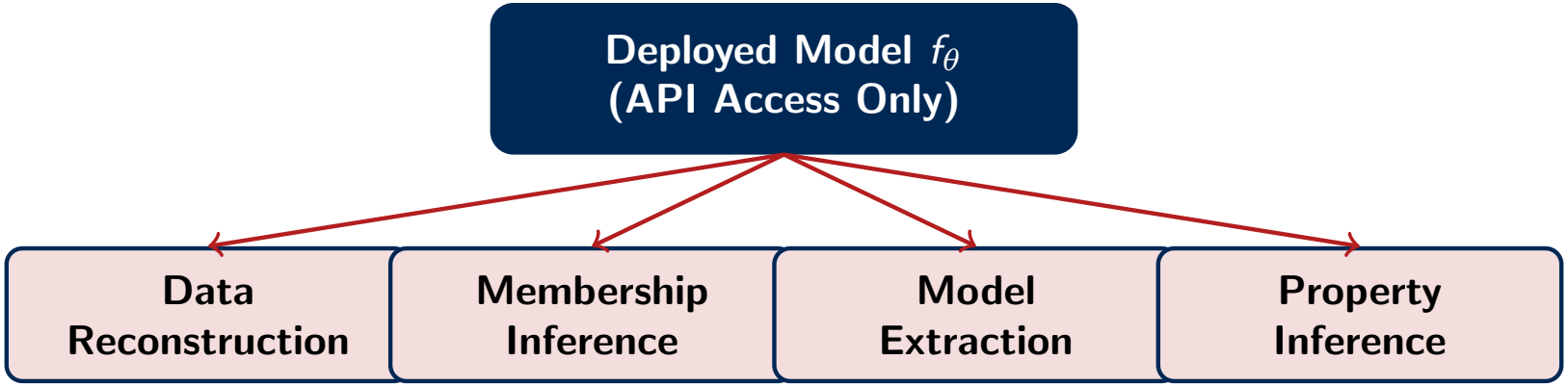
# Privacy Attacks: Four Classes (NIST AI 100-2)

NIST AI 100-2 organizes privacy attacks for PredAI into four classes, all exploitable through a model's *public API alone* [1]:

1. **Data reconstruction**: recover specific training inputs from model outputs
2. **Membership inference**: determine whether a given record was in the training set
3. **Model extraction**: replicate the model's decision function via systematic API queries
4. **Property inference**: infer aggregate statistics of training data (e.g., class balance, demographic composition)

**Hallmark insight**: an ML model is not a black box – it is a compressed, queryable summary of its training data. Adversaries exploit that summary.

# Privacy Attacks: Four Classes (NIST AI 100-2)




All four attacks require only API query access.

# Membership Inference: Theory and Attack

## Membership Inference Attack

Given a trained model $f$ and a sample $(\mathbf{x}, y)$, determine: **was $(\mathbf{x}, y)$ used to train $f$?**

Key insight: overfit models assign *higher confidence scores* to training members than to unseen samples. This **confidence gap** is the exploitable signal.

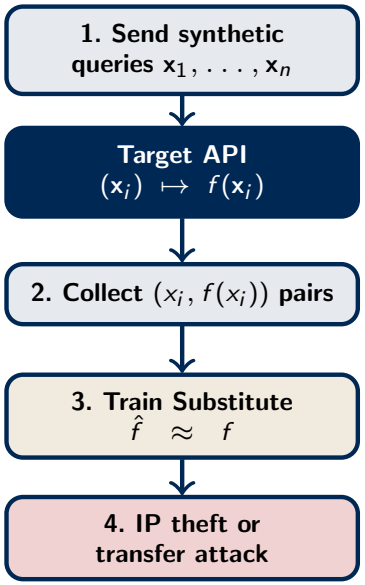
**Shokri et al. (2017) shadow model attack:**

1. Train multiple **shadow models** that mimic the target
2. Collect (confidence vector, member/non-member) labels from shadow models
3. Train a binary **meta-classifier** on those labels
4. Apply the meta-classifier to target model outputs

# Model Extraction: Stealing Models via APIs

## Model Extraction / Model Stealing

An adversary sends chosen queries $\{\mathbf{x}_i\}$ to a target model API and collects responses $\{f(\mathbf{x}_i)\}$. Using these pairs, the adversary trains a **substitute model** $\hat{f}$ that approximates the target.




**Attack objectives:**

- **IP theft**: replicate proprietary functionality at far lower cost
- **Attack enablement**: run white-box FGSM/PGD on the substitute, then transfer to the target
- **Regulatory evasion**: deploy the stolen model without the compliance controls of the original provider

# Data Reconstruction and Training Data Extraction

## Training Data Extraction

For generative models: an adversary uses specially crafted prompts to cause the model to **reproduce verbatim training data**. For discriminative models: **model inversion** uses output gradients to reconstruct the average training input for a class.

## Differential Privacy (DP)

DP training adds calibrated noise to gradients, bounding each sample's influence:

$$\Pr[M(\mathcal{D}) \in S] \leq e^{\varepsilon} \Pr[M(\mathcal{D}') \in S] + \delta$$

for adjacent datasets $\mathcal{D}, \mathcal{D}'$ differing by one record.

**Carlini et al. (2023) — ChatGPT extraction:**

- Prompt: "Repeat the word 'poem' forever" (divergence trigger)
- Verbatim text extracted at ∼1-in-10 query success rate
- Recovered names, email addresses, phone numbers, memorized code
- Total cost: under $200 in API credits [11]

# Evasion Attacks: Fooling Deployed Models

**Evasion Attack**

A **deployment-time** attack that adds a small perturbation $\boldsymbol{\delta}$ to an input $\mathbf{x}$ such that $f(\mathbf{x} + \boldsymbol{\delta}) \neq f(\mathbf{x})$, while $\|\boldsymbol{\delta}\|_p \leq \epsilon$.

## Key properties:

- Constrained by perturbation budget $\epsilon$ under $L_\infty$, $L_2$, or $L_0$ norms
- **Untargeted**: any misclassification suffices
- **Targeted**: force a specific class $y_t$
- **Transferable**: adversarial examples crafted on one model often fool a different model
- Apply across all modalities: images, text, audio, **tabular data**

**Tabular evasion**: manipulating features to cross a decision boundary (inflating income, reducing stated debt) is structurally equivalent to FGSM.

# PGD and Evasion Defenses

## PGD — Madry et al. (2018)

Iterative projected gradient descent; gold standard:

$$\mathbf{x}^{(t+1)} = \Pi_{\mathcal{B}_\epsilon(\mathbf{x})}\big(\mathbf{x}^{(t)} + \alpha \cdot \text{sign}(\nabla_{\mathbf{x}}\mathcal{L})\big)$$

$\Pi$ projects back into the $\epsilon$-ball; random restarts prevent local minima.

# PGD and Evasion Defenses

## Evasion Defenses

**Proactive:**

- **Adversarial training** (Madry et al.): augment training with PGD examples; most effective but trades 2–10 pp clean accuracy for robustness
- **Randomized smoothing**: certified $L_2$ robustness via Gaussian noise
- **Input preprocessing**: JPEG compression, feature squeezing, denoising autoencoders

**Reactive:**

- Statistical anomaly detection on input distributions
- Prediction inconsistency across perturbations

# ⋆ 5-Minute Break ⋆

We resume with the Live Coding Lab

# Lab Overview: What We Will Build

**Objective**: implement four full adversarial attack pipelines on COMPAS, continuing directly from the Lecture 04 codebase.

**Source code**: instructor GitHub link

**Pipeline components:**

1. **Setup** (Cells 1–2): Load COMPAS, retrain LR and GBT from Lecture 04; establish clean-model fairness baseline (FPR and AIR by race)
2. **Part A — PGD Evasion** (Cells 3–4): Iterative projected gradient descent on tabular features; disparate impact curve by race vs. epsilon
3. **Part B — Poisoning Loop** (Cells 5–6): Label-flip poisoning with varying poison rate; joint AUC and AIR degradation plot
4. **Part C — Membership Inference** (Cells 7–8): Full shadow-model pipeline; ROC curve and confidence gap plot

# Lab Overview: What We Will Build

## Lab Findings to Discuss

- **PGD**: does the racial FPR gap widen or narrow as $\epsilon$ increases?
- **Poisoning**: at what rate does AUC remain stable while AIR falls below 0.80?
- **MI**: is MI AUC correlated with the generalization gap? What does that imply for DP adoption?

# Quiz Time (10 Minutes)

Justify all claims with reference to lecture content

# Lecture Summary

## NIST AI 100-2 Attack Taxonomy

**Training-time:**

- Availability / targeted / clean-label poisoning
- Backdoor (trigger-to-target; survives fine-tuning)
- Supply chain: Trojan pre-trained models

**Deployment-time:**

- Evasion: PGD; transfer attacks
- Membership inference: confidence gap exploitation
- Model extraction: active-query substitute training

# Lecture Summary

## Core Principle

A model fragile to distribution shift (Lecture 04) shares the same root cause as a model vulnerable to adversarial attack: over-reliance on brittle, non-robust features. Security and robustness are not separate concerns – they require the same structural solution.

# Group Homework 05 – Theoretical, Due at 11:59pm ET on April 30, 2026

Using the NIST AI 100-2 taxonomy as your framework, answer the following questions analytically. **No code required.** Cite relevant literature to support each claim.

1. **Threat Modeling**: Identify *three* realistic adversarial actors in the COMPAS deployment pipeline (e.g., defendant, data clerk, jurisdiction administrator, third-party vendor). For each actor, specify: **(a)** their goal (availability, integrity, or privacy breach); **(b)** their knowledge level (white/gray/black-box); **(c)** what they can control; and **(d)** the NIST AI 100-2 attack class that best describes their attack.

# Group Homework 05 – Theoretical, Due at 11:59pm ET on April 30, 2026

2. **Privacy–Fairness Tradeoff**: Bagdasaryan & Shmatikov (2019) showed that differential privacy disproportionately harms minority subgroups. **(a)** State the $(\varepsilon, \delta)$-DP guarantee formally. **(b)** Explain the mechanism by which DP training degrades minority-group accuracy more than majority-group accuracy. **(c)** In the COMPAS context, discuss whether a county probation office can simultaneously satisfy DP, FPR parity, and membership inference resistance. Is this theoretically possible? Under what conditions?
3. **Governance Recommendation**: Draft a one-paragraph policy recommendation for a court system deploying COMPAS, specifying which attack class poses the greatest risk, which mitigation you recommend, and what tradeoff it introduces.

*Deliverable: typed PDF, max 4 pages.*

# Individual Homework 05 – Applied, Due at 11:59pm ET on April 30, 2026

Using the ProPublica COMPAS dataset and the live-coding notebook from this lecture as your starting point.

1. **PGD Evasion Audit**: Run the PGD attack across $\varepsilon \in \{0.25, 0.5, 1.0, 2.0\}$ on both the LR and GBT models. For each model and $\varepsilon$: report FPR by race, AIR, and the $\varepsilon$ at which AIR crosses 0.80. In one paragraph, explain whether the two models are equally vulnerable and what this implies for model selection in high-stakes deployments.

# Individual Homework 05 – Applied, Due at 11:59pm ET on April 30, 2026

2. **Poisoning Loop with Fairness Monitoring**: Extend the label-flip poisoning loop to target `Caucasian` defendants instead of `African-American`. **(a)** Plot AUC and AIR degradation curves for both target-race variants on the same axes. **(b)** Identify the "stealth zone": the range of poison rates where AUC declines by $\leq 2$ pp while AIR moves outside [0.80, 1.25]. **(c)** Would a PSI-based drift monitor (threshold $< 0.10$ on each feature) detect either attack? Justify with evidence from your output.

# Individual Homework 05 – Applied, Due at 11:59pm ET on April 30, 2026

3. **Membership Inference Depth**: Compute the shadow-model MI AUC for both the LR and GBT models. **(a)** Plot the confidence-gap histogram for each model side by side. **(b)** Test whether the generalization gap (train AUC − test AUC) predicts the MI AUC across the two models. **(c)** Add $L_2$ regularization to the LR model (vary $C \in \{0.01, 0.1, 1.0, 10.0\}$), recompute MI AUC, and plot MI AUC vs. $C$. What is the practical tradeoff?
4. **Reflection**: Identify your single highest-risk finding across all three parts. Propose one proactive and one reactive mitigation, quantify each mitigation's effect from your experimental results, and discuss any disparate impact the mitigations introduce on either racial group.

*Deliverable: Colab notebook + 2-page written report.*

# Homework Evaluation Criteria

**Group homework**

- conceptual precision
- rigor of metric-to-governance mapping
- quality of sociotechnical reasoning
- clarity and defensibility of recommendations

**Individual coding homework**

- correctness of implementation
- code clarity and reproducibility
- interpretation quality
- evidence of audit-level reasoning

# Homework Evaluation Criteria

**What earns a strong grade**

Do not just report metrics. Explain what they mean, what they miss, and what actions they justify.

# References


[1] Vassilev A, Oprea A, Fordyce A, Anderson H (2024). *Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations*. NIST AI 100-2e2023. https://doi.org/10.6028/NIST.AI.100-2e2023

[2] NIST (2025). *Adversarial Machine Learning: A Taxonomy and Terminology*. NIST AI 100-2e2025. https://csrc.nist.gov/pubs/ai/100/2/e2025/final

[3] NIST (2023). *AI Risk Management Framework (AI RMF 1.0)*. NIST AI 100-1. https://doi.org/10.6028/NIST.AI.100-1

[4] Biggio B, Roli F (2018). Wild Patterns: Ten Years After the Rise of AML. *Pattern Recognition*, 84. https://arxiv.org/abs/1712.03141

[5] Hall P, Curtis J, Pandey P (2023). *Machine Learning for High-Risk Applications*, Ch. 5. O'Reilly. https://www.oreilly.com/library/view/machine-learning-for/9781098102425/

[6] Goodfellow IJ, Shlens J, Szegedy C (2015). Explaining and Harnessing Adversarial Examples. *ICLR 2015*. https://arxiv.org/abs/1412.6572

# References


[7] Madry A et al. (2018). Towards Deep Learning Models Resistant to Adversarial Attacks. *ICLR 2018*. https://arxiv.org/abs/1706.06083

[8] Carlini N, Wagner D (2017). Towards Evaluating the Robustness of Neural Networks. *IEEE S&P 2017*. https://arxiv.org/abs/1608.04644

[9] Shokri R et al. (2017). Membership Inference Attacks Against ML Models. *IEEE S&P 2017*. https://arxiv.org/abs/1610.05820

[10] Tramer F et al. (2016). Stealing Machine Learning Models via Prediction APIs. *USENIX Security 2016*. https://arxiv.org/abs/1609.02943

[11] Nasr M et al. (2023). Scalable Extraction of Training Data from (Production) Language Models. https://arxiv.org/abs/2311.17035

[12] Eykholt K et al. (2018). Robust Physical-World Attacks on Deep Learning. *CVPR 2018*. https://arxiv.org/abs/1707.08945

[13] Bagdasaryan E, Shmatikov V (2019). Differential Privacy Has Disparate Impact. *NeurIPS 2019*. https://arxiv.org/abs/1905.12101

# References

[14] Refonte Learning (2025). Protect Your AI Models from Adversarial Attacks. https://www.refontelearning.com/blog/protect-your-ai-models-from-adversarial-attacks-advanced-strategies-fo

[15] SecureWorld (2025). How Real Is the Threat of Adversarial AI? https://www.secureworld.io/industry-news/how-real-threat-adversarial-ai

# Grading and Policy

- Grading
  - Weekly assignment: $\frac{2.5}{10}$
  - Public GitHub with working codebase: $\frac{2.5}{10}$
  - Weekly quiz: $\frac{2}{10}$
  - Capstone: $\frac{3}{10}$
- Coding/Scripting
  - Python
  - R
- Collaborative Infrastructure
  - GitHub

# Grading and Policy

- Capstone project
  - 2024 Home Mortgage Disclosure Act (HMDA): Loan/Application Records (LAR) data
  - Public LAR data: dictionary
  - Goal: Develop a classification model that predicts `action_taken` and apply principles taught in class to the model.
  - Label creation: `action_taken` values 1 and 2 coded as 1 and 3 coded as 0 while records matching other values are filtered out.
  - Group (no more than 5 members and no less than 3 members)
  - Group members are randomly assigned by instructor using the standard *math* and *random* libraries in **Python**.
- Syllabus: link
- Officer hours: Funger Hall 415R, Fri, 4:00PM - 6:00PM
- Class resources:
  - Instructor's materials
  - Additional resources from Patrick Hall